UML Practice Exercises

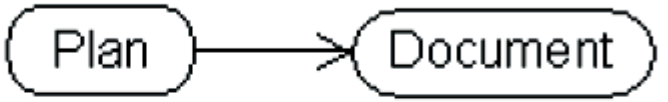
Exercise 1

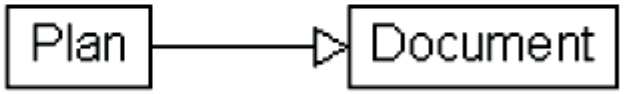
Match the UML model with the appropriate interpretation.

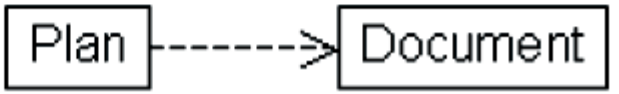
___ A plan is dependent on a document.

___ Plan first, then document.

___ A plan is a special type of document.

A. 

B. 

C. 

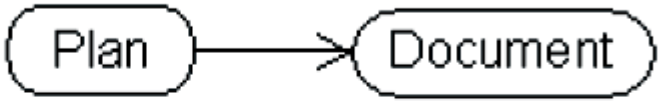
Exercise 1

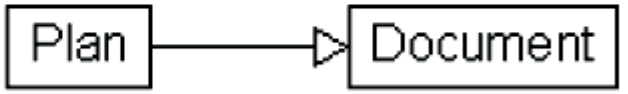
Match the UML model with the appropriate interpretation.

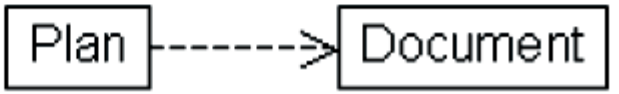
___ A plan is dependent on a document.

___ Plan first, then document.

___ A plan is a special type of document.

A. 

B. 

C. 

Solution 1

Ans. C, A, B

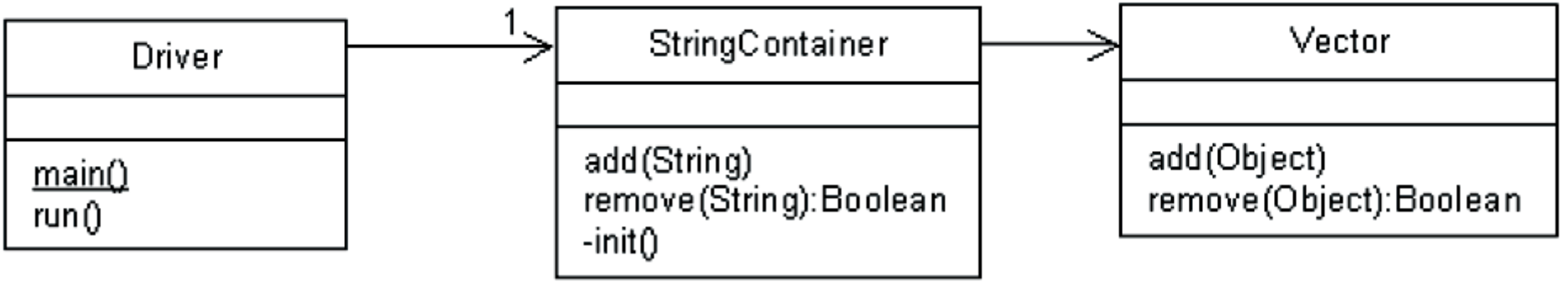
Exercise 2

```
import java.util.Vector;

public class Driver {
    private StringContainer b = null;

    public static void main(String[] args){
        Driver d = new Driver();
        d.run();
    }

    public void run() {
        b = new StringContainer();
        b.add("One");
        b.add("Two");
        b.remove("One");
    }
}

class StringContainer {
    private Vector v = null;

    public void add(String s) {
        init();
        v.add(s);
    }

    public boolean remove(String s) {
        init();
        return v.remove(s);
    }

    private void init() {
        if (v == null)
            v = new Vector();
    }
}
```

Draw an UML class diagram to express the structural relationships in the above program and draw an UML sequence diagram to express the dynamic behavior.

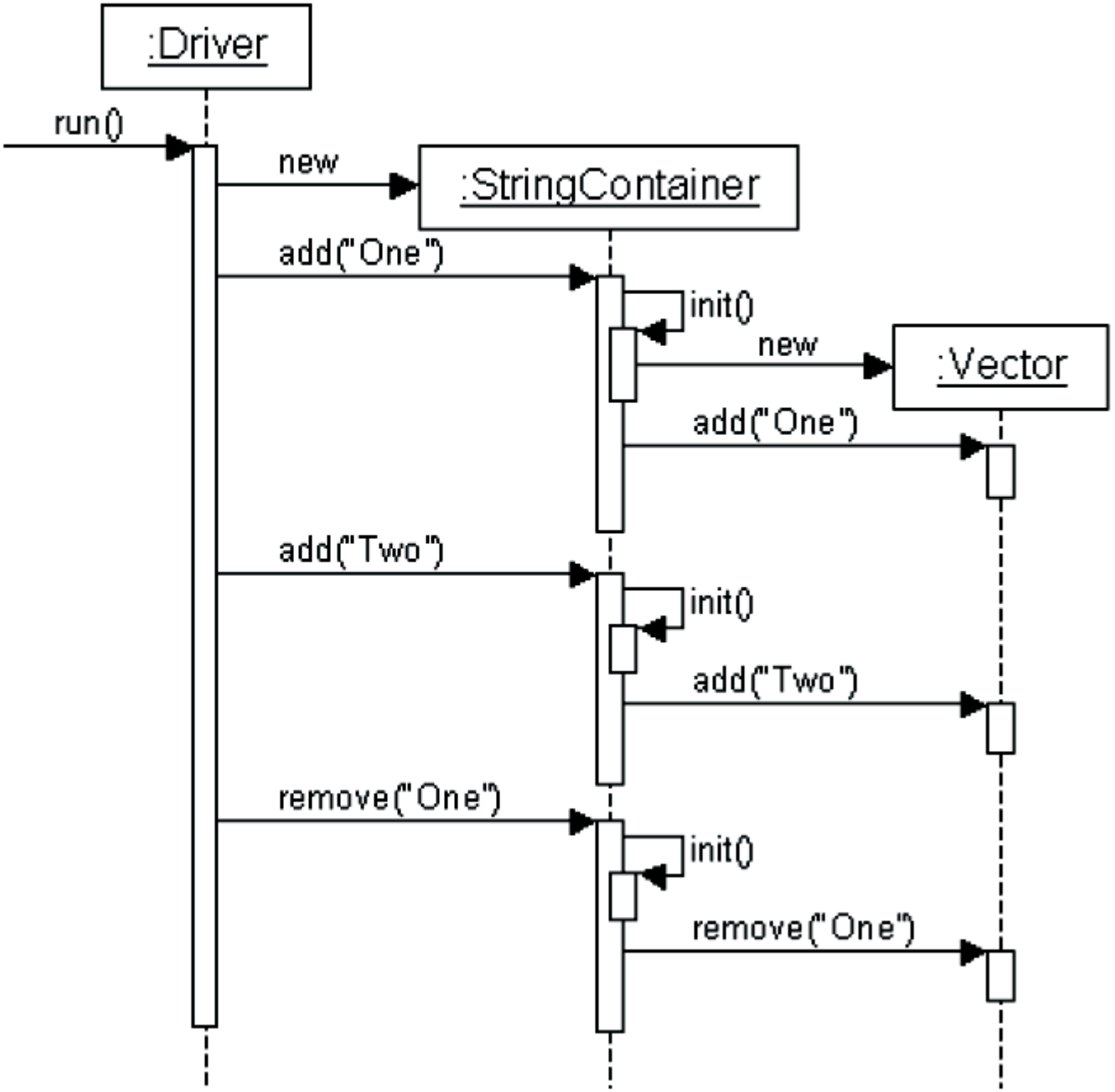
Solution 2

Class Diagram:



Solution 2 ... cont'd

Sequence Diagram:

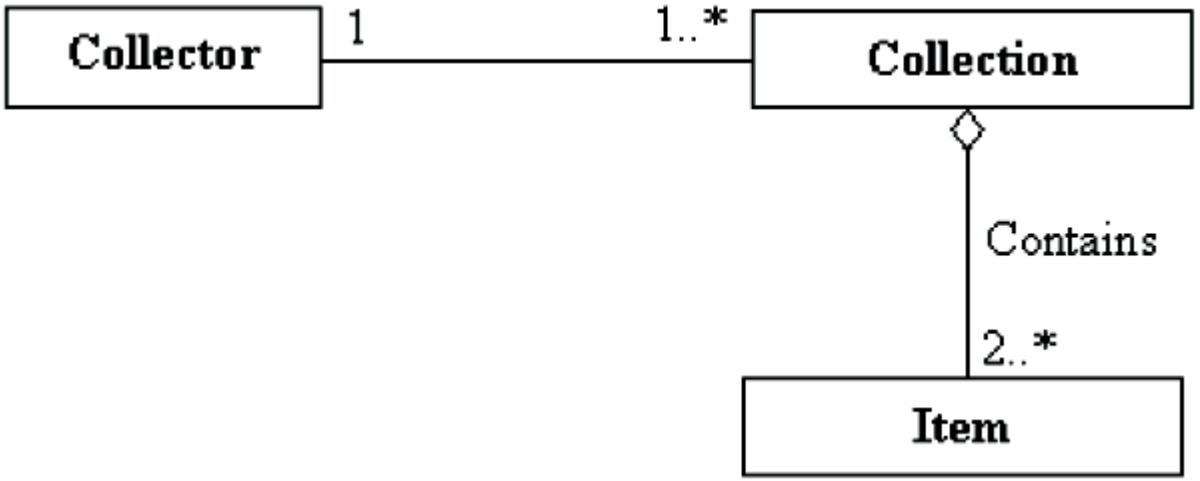


Exercise 3

Create an UML class diagram that models the data relationships described in the following paragraph.

> To be a collector you have to have one or more collections. Each collection must have 2 or more items. Each collection belongs to one collector. A collection is made up of items owned. A particular item may be in more than one collection (i.e. an old Coke sign may be in both a Coke memorabilia collection and a sign collection.)

Solution 3



Exercise 4

Use the following class diagram to answer the questions that following it. If there isn't enough information in the diagram to answer the question, state as much.

1. Can you have a company without any employees?
2. Can a person be unemployed?
3. Can a person be self-employed?

Solution 4

1. No, the multiplicity on the employee side is 1..*.
2. Yes, the multiplicity on the employer side is * so 0 is a valid value.
3. According to the diagram if a person was self-employed that person would have to work for the company he or she owned.

Exercise 5

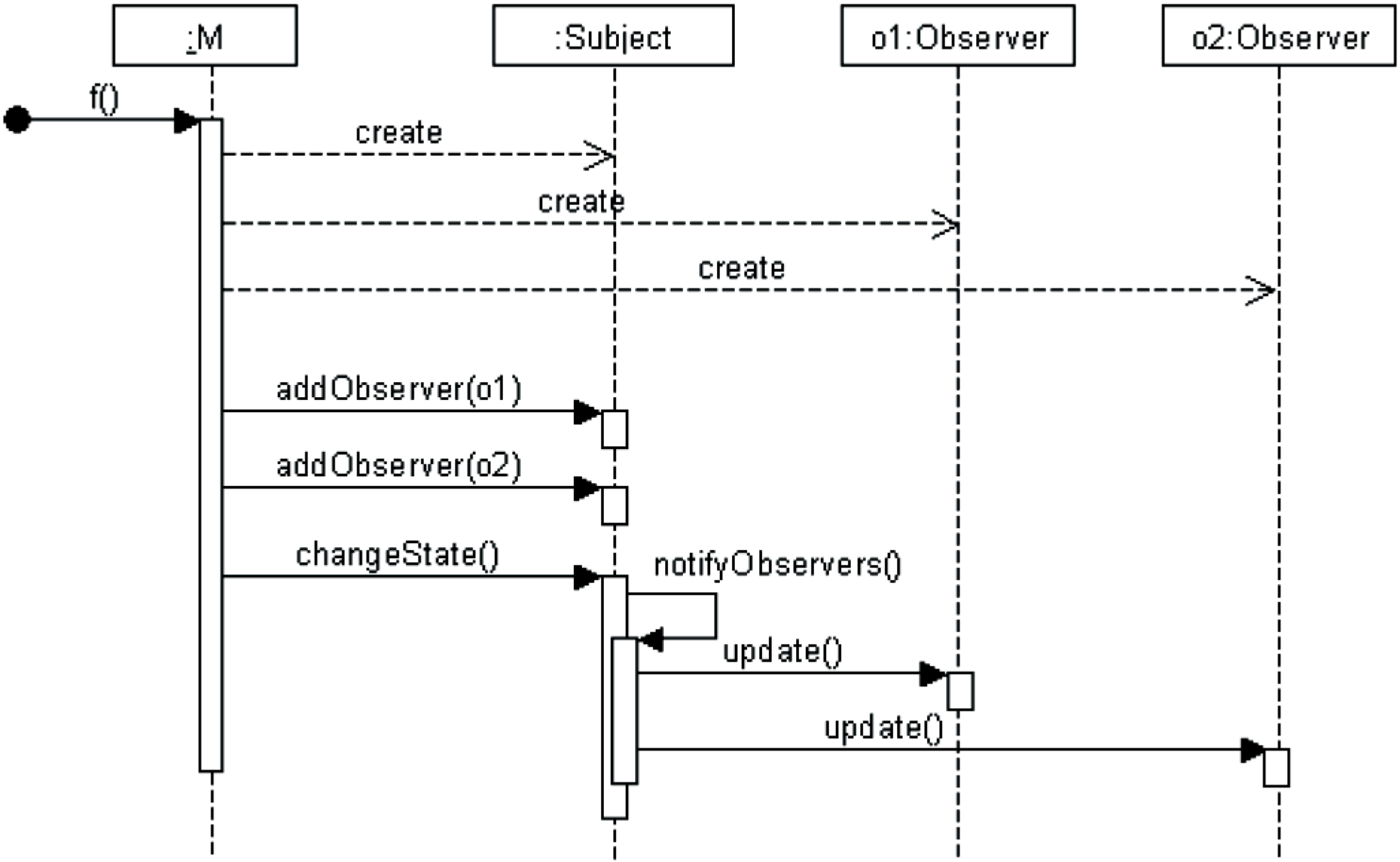
Draw the UML class diagram and UML sequence diagram for the following code fragments. You can assume class M already exists and start the sequence diagram with the method f(). You don't have to document calls to any methods defined for classes not shown here.

```
public class M {

  public static void main(String[] args) {
    M m = new M();
    m.f();
  }
  public void f() {
    Subject s = new Subject();
    Observer o1 = new Observer();
    Observer o2 = new Observer();
    s.addObserver(o1);
    s.addObserver(o2);

    s.changeState();
  }
}
```

```
class Observer {
    public void update(){}
}
```

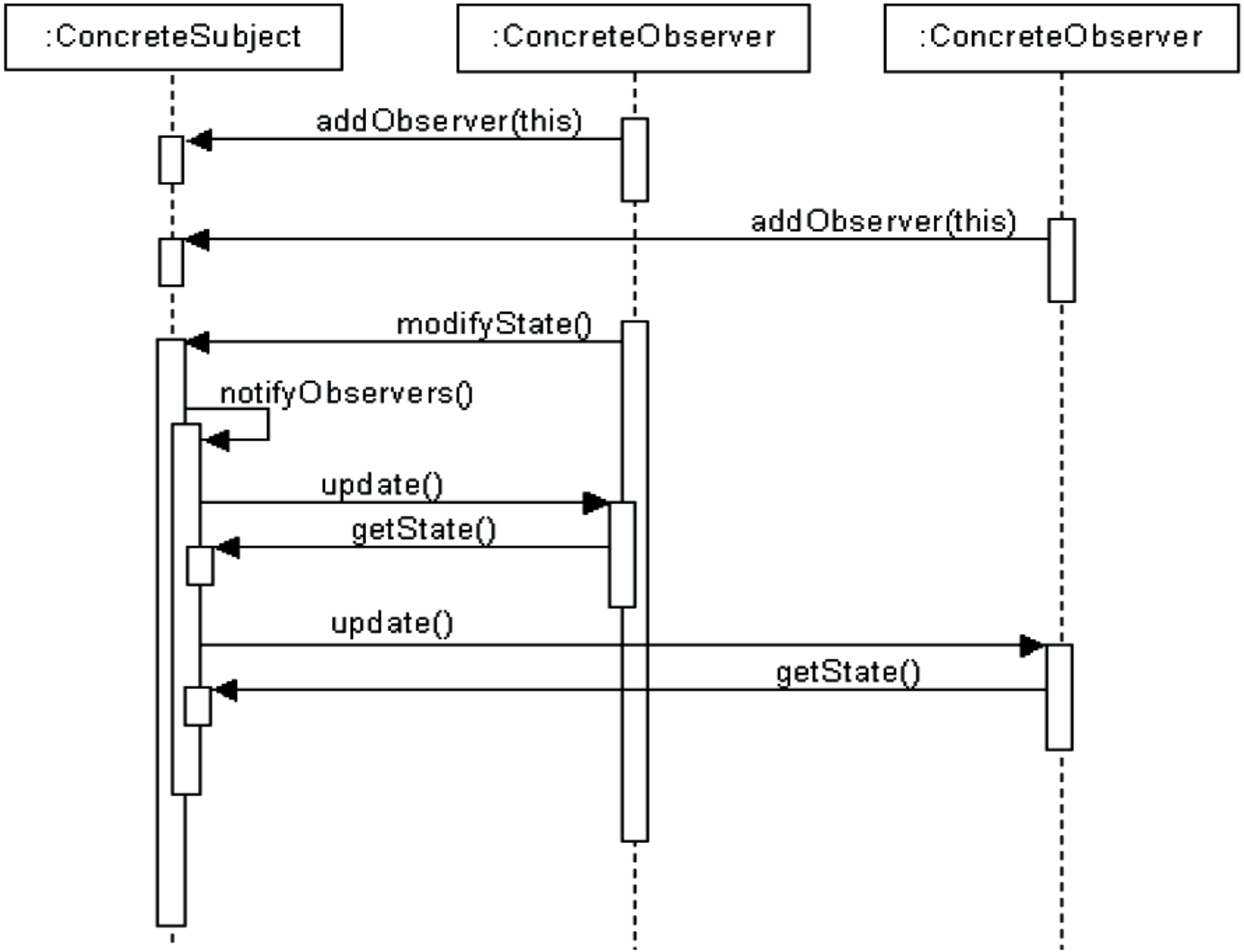
```
class Subject {
  private Collection c;

  public void addObserver(Observer o) {
    c.add(o);
  }

  public void changeState() {
    // Change state of subject
    notifyObservers();
  }

  public void notifyObservers() {
    Iterator i = c.iterator();
    while (i.hasNext()) {
      Observer o = (Observer)i.next();
      o.update();
    }
  }
}
```

Solution 5


:M
:Subject
o1:Observer
o2:Observer
f()
create
create
create
addObserver(o1)
addObserver(o2)
changeState()
notifyObservers()
update()
update()

Exercise 6

Write the code fragment that corresponds to the following sequence diagram.



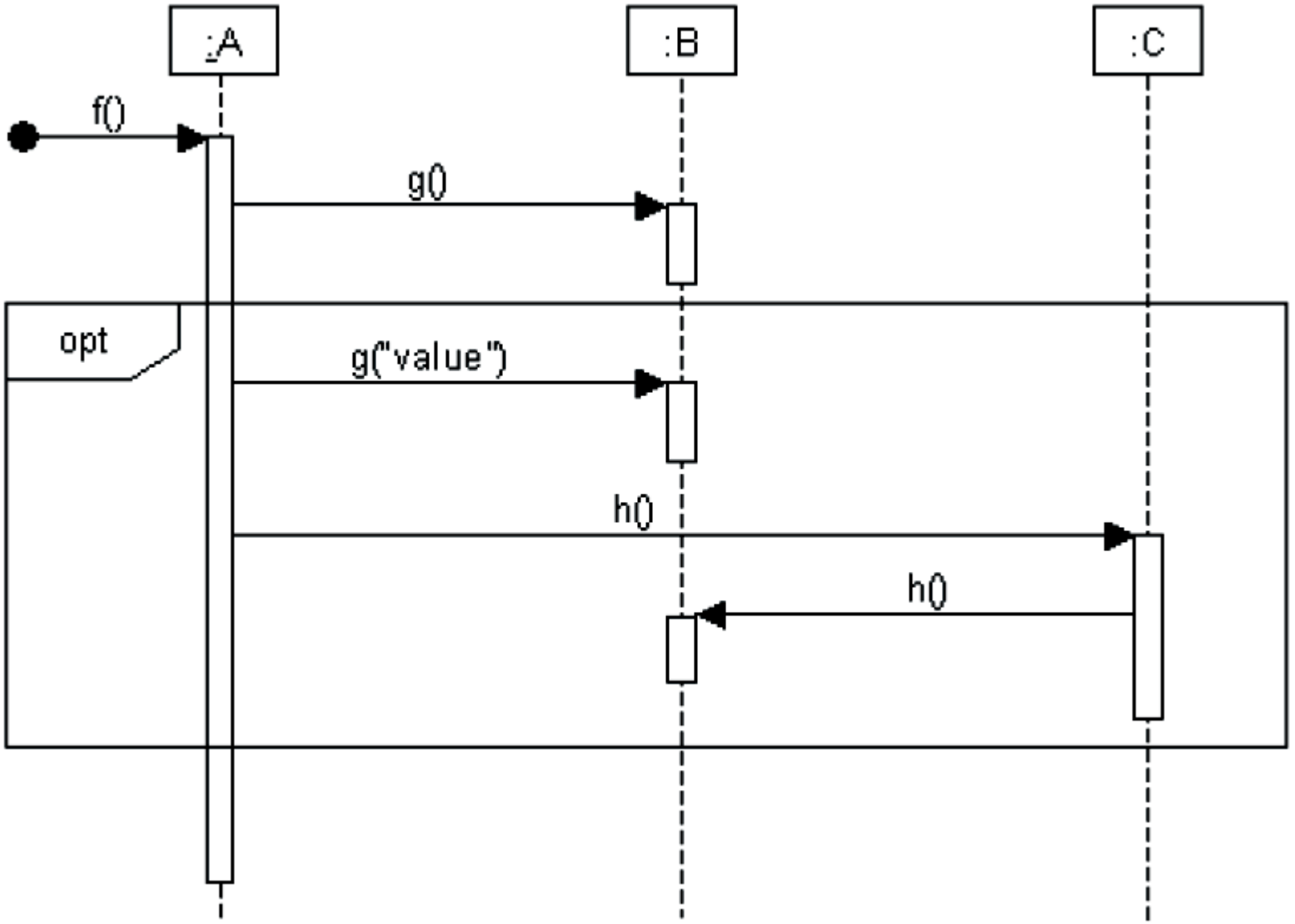
Solution 6

While incomplete, code fragment below reflects essence of the Seq. diagram above.

```
public class ConcreateSubject {

    public void addObserver(ConcreateObserver o) {...}

    public void modifyState() {
        notifyObservers();
    }

    public void notifyObservers() {
        for each observer o {
            o.update(this);
        }
    }

    public Object getState() { ...}
}

public class ConcreteObserver {

    public void update(ConcreateSubject s) {
        s.getState();
    }
}
```

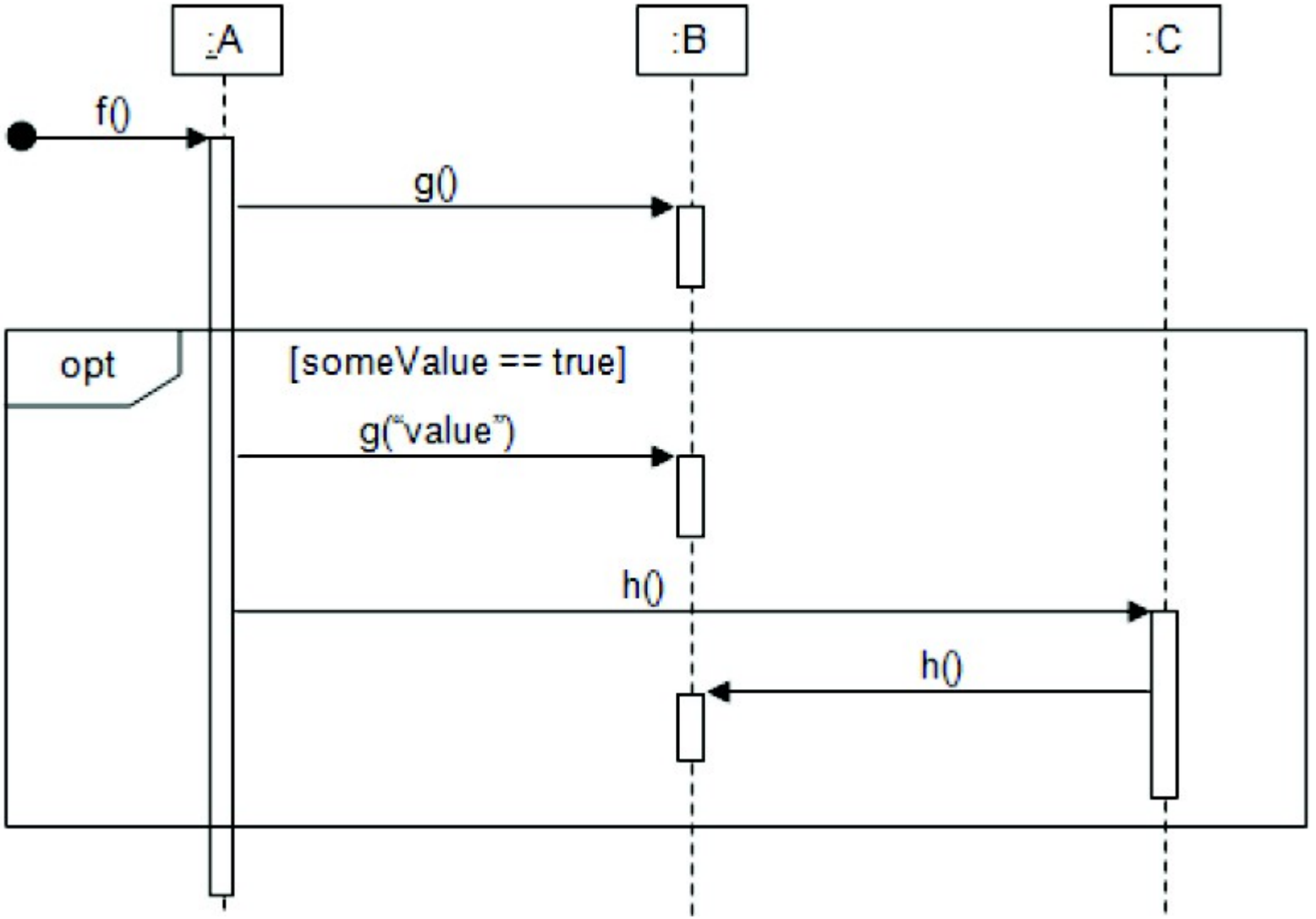
Exercise 7

Is there anything wrong with or missing from the following sequence diagram? If so, what is wrong or missing? List all with reasons.

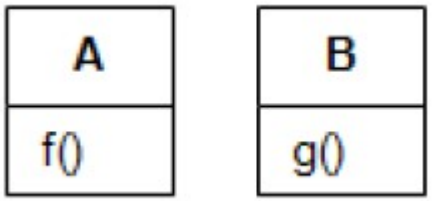


Solution 7

- The sequence diagram above includes a combined fragment with the opt ("optional") interaction operator but doesn't include a guard condition saying when the optional fragment should be executed.
- It should include a guard condition of the form: [boolean expression]. The optional fragment executes only if the boolean condition is true.

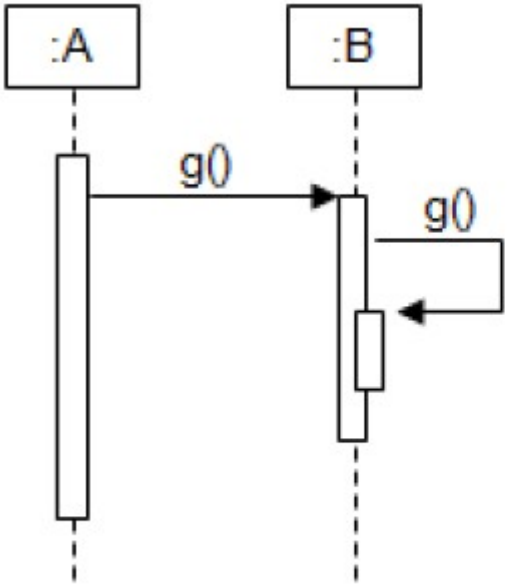


Exercise 8

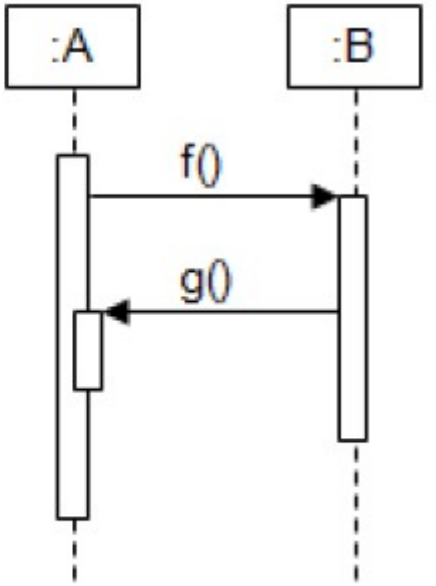


Which of the following sequence diagrams are potentially valid for the above given class diagram?

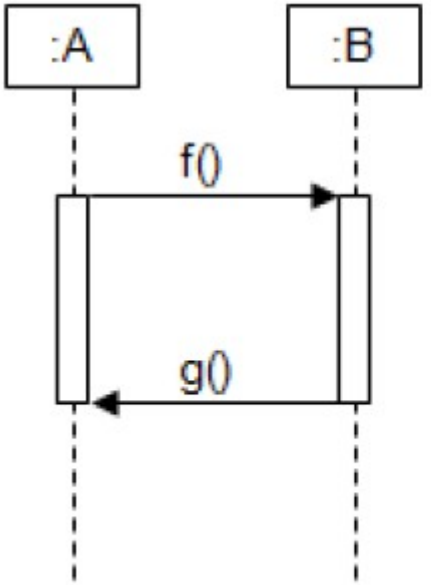
Check all that apply.



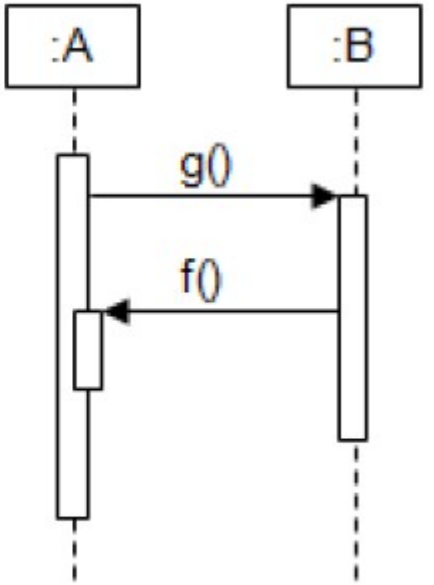
(a)



(b)



(c)

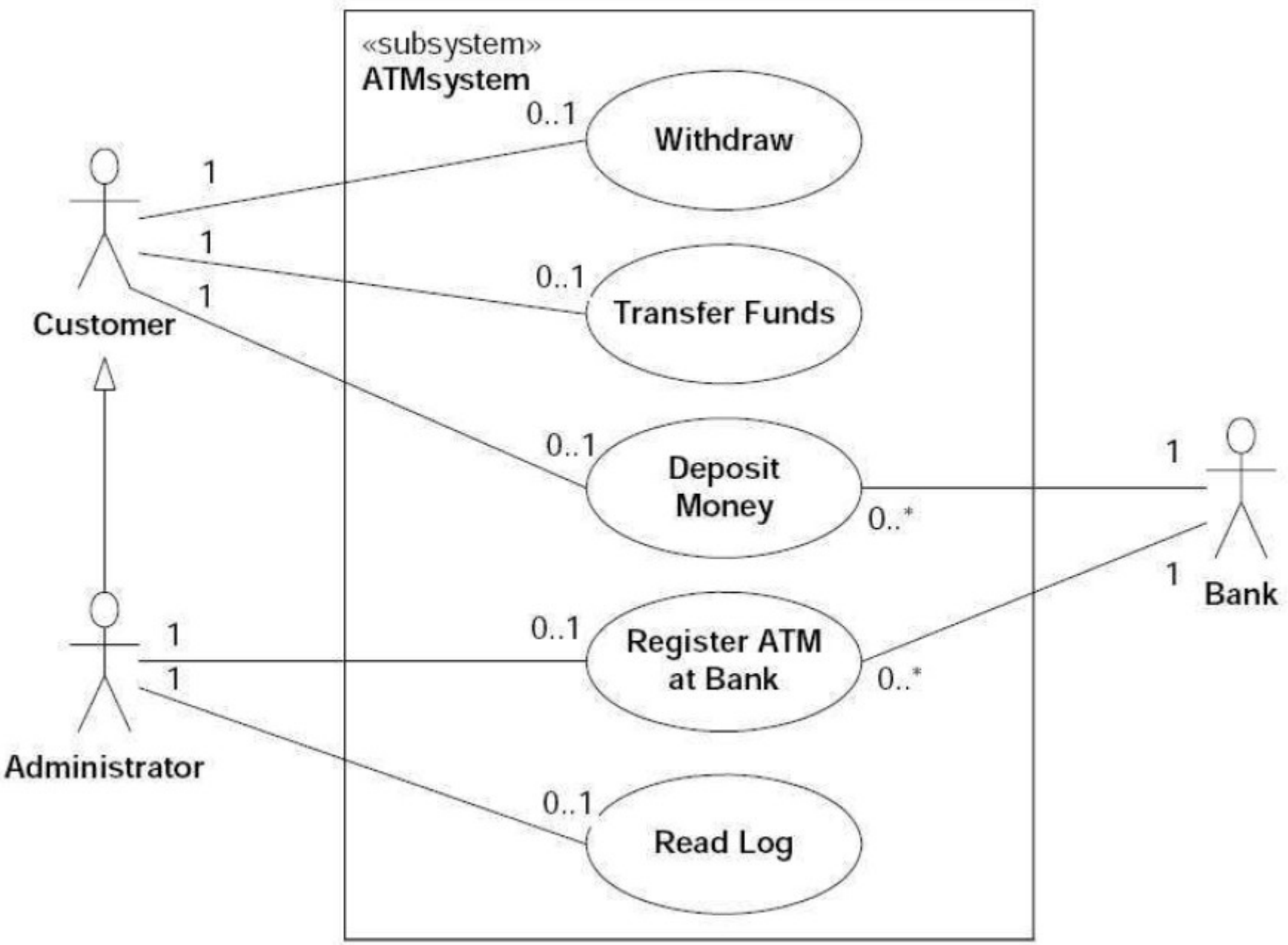


(d)

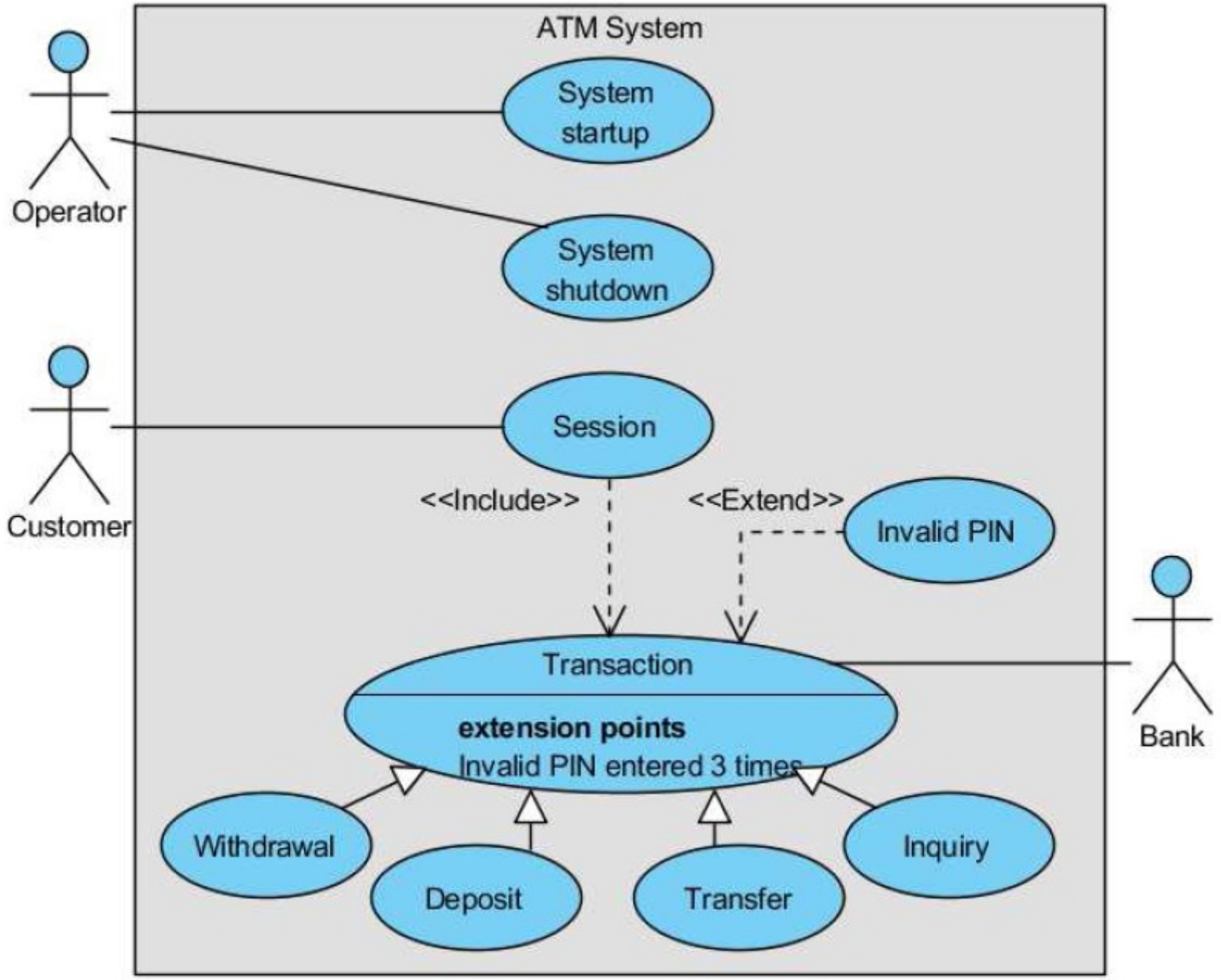
Exercise 9

- We know the working of a bank ATM system. You are requested by your client to capture the working of ATM system using a UML design model.
  - Identify relevant use cases and represent them using a suitable Use-case diagram. Also, give the narrative version of all use-cases.
  - Represent the different identified scenarios using suitable sequence diagrams.
  - Represent the class diagram for this ATM system

Use-case diagram of ATM system



Another Use-case diagram of ATM system

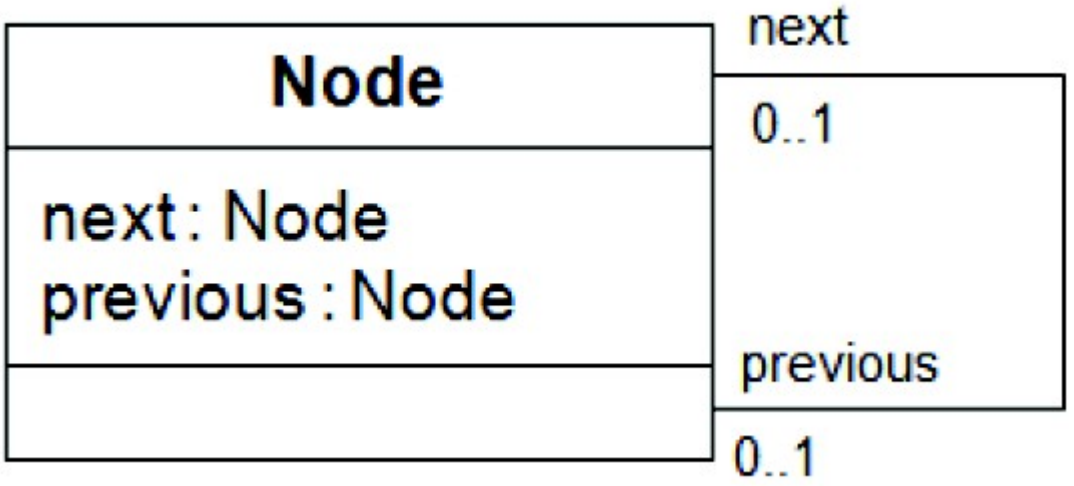


Exercise 10

- Draw a class diagram representing a singly linked list.

- **Hint:** Both ends may attach to the same classifier when one instance refers to another instance or when one instance refers to itself. For example, consider a node in a linked list (see next slide)

Solution 10

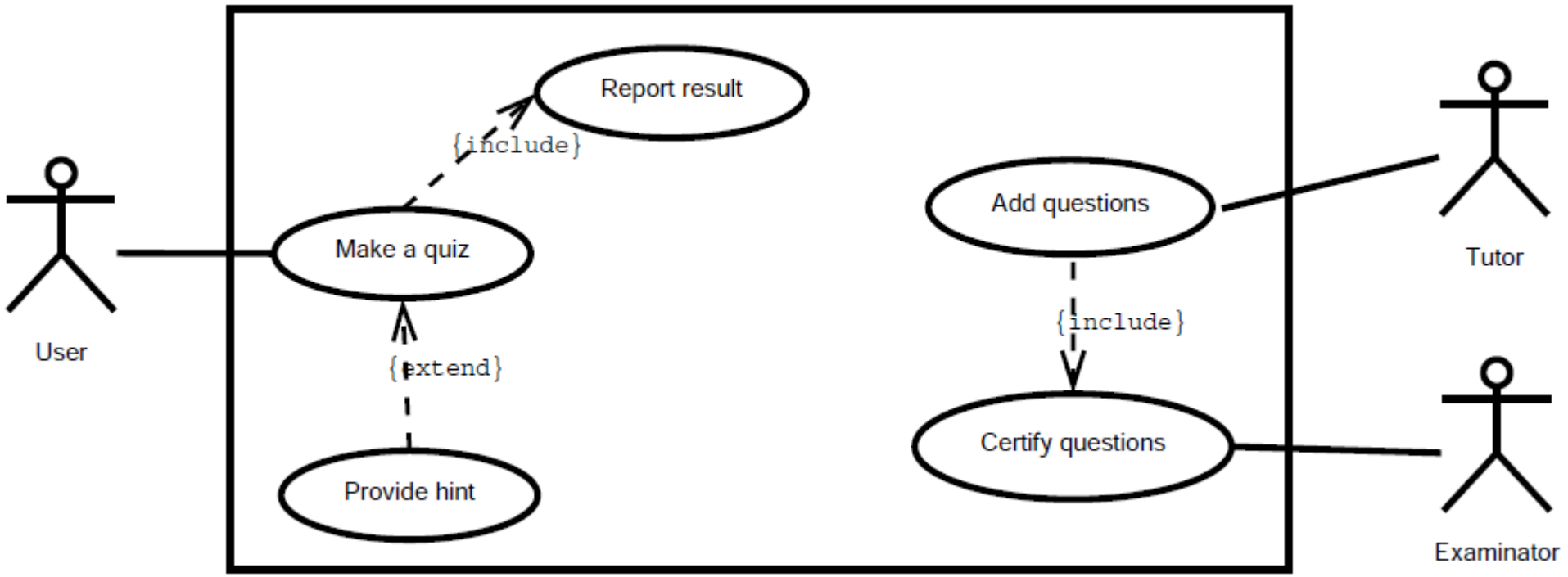


Exercise 11

- Let’s imagine we want to develop a browser-based training system to help people prepare for ’Certified Java Programmer’ like certification exam.
  - A user can request a quiz for the system.
  - The system picks a set of questions from its database, and compose them together to make a quiz.
  - It rates the user’s answers, and gives hints if the user requests it.
  - In addition to users, we also have tutors who provide questions and hints.
  - And also examinators who must certify questions to make sure they are not too trivial, and that they are sensical.

Make a use case diagram to model the above system. Work out some of your use cases. You are free to fill in details you think are useful for giving your UML model.

Solution 11


Report result
{include}
Make a quiz
{extend}
Provide hint
Add questions
{include}
Certify questions
User
Tutor
Examinator

Solution 11 ... cont'd

Use case: Make quiz.

Actor: User **Pre-condition:** The system has at least 10 questions.

Main flow:

1. The use-case is activated when the user requests it.

2. The user specifies the difficulty level.

3. The system selects 10 questions, and offers them as a quiz to the user.

4. The system starts a timer.

5. For every question, the user selects an answer, or skip. [Extension point]

6. If the user is done with the quiz, or the timer runs out, the quiz is concluded, and [include use case 'Report result'].

Use case: Provide hint

Actor: User **Pre-condition:** The user requests for a hint.

Main flow:

1. The system provides a hint. The verbosity of the hint is determined by the difficulty level set previously by the user.

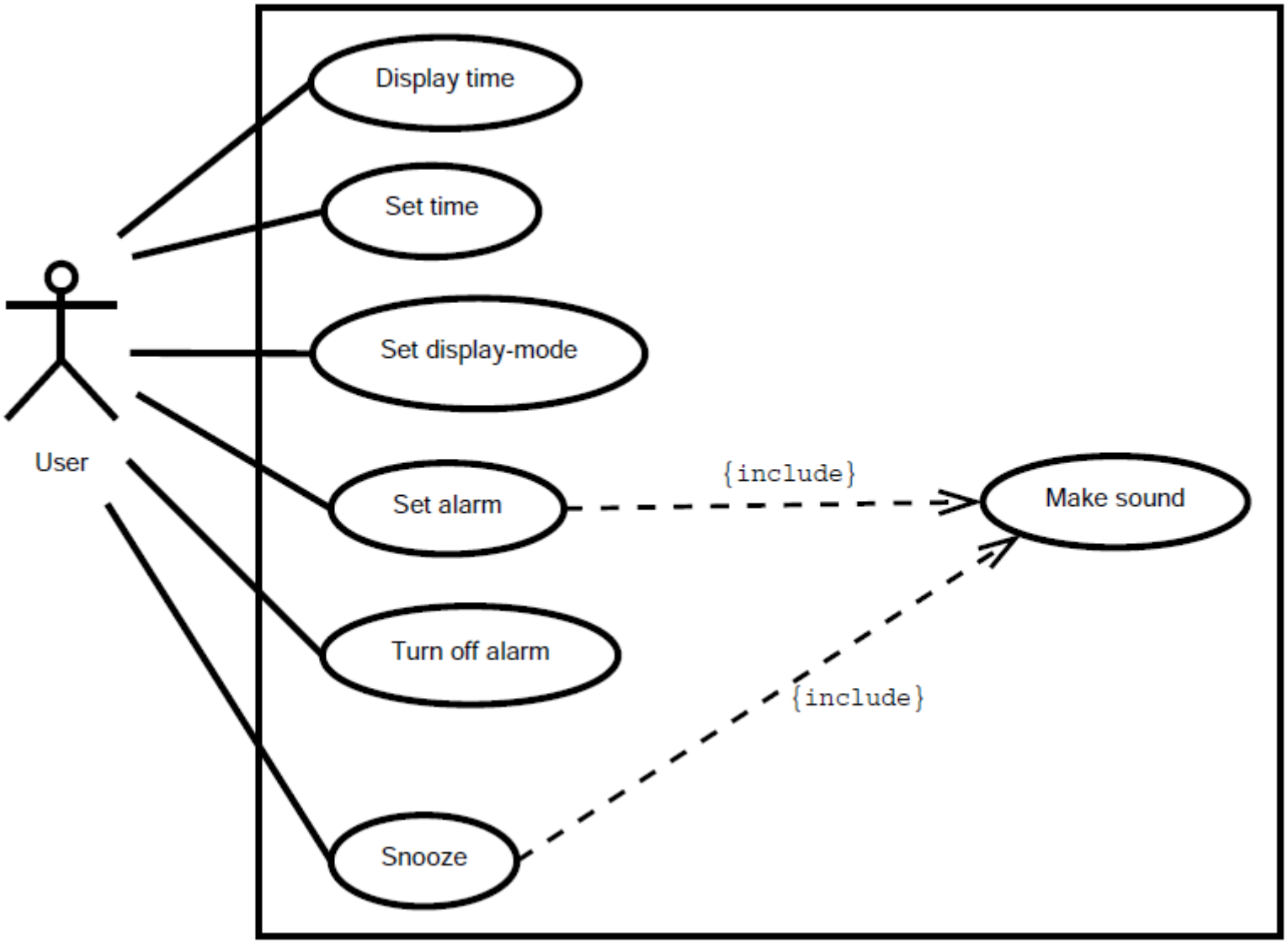
2. Return to Make quiz' main flow.

Exercise 12

- Suppose we want to develop software for an alarm clock.
- The clock shows the time of day. Using buttons, the user can set the hours and minutes fields individually, and choose between 12 and 24-hour display.
- It is possible to set one or two alarms. When an alarm fires, it will sound some noise. The user can turn it off, or choose to ’snooze’. If the user does not respond at all, the alarm will turn off itself after 2 minutes. ’Snoozing’ means to turn off the sound, but the alarm will fire again after some minutes of delay. This ’snoozing time’ is pre-adjustable.

- Identify the top-level functional requirement for the clock, and model it with a use case diagram.

Solution 12



- In this model ’Make sound’ is made as a separate use case. It does not have to.
- I assume that ’snooze’ would be one of your use cases. Work it out to the fully dressed level.

Solution 12 ...cont'd

Use case: Snooze.

Primary actor: User

Pre-condition: An alarm is firing.

Main flow:

1. The use-case is activated when the user hits the snooze button.
2. The alarm is turned off.
3. Wait for snooze time.
4. Include use case ’Make sound’

Use case: Make sound

Primary actor: System

Main flow:

The use case starts when it is called. What it does is to just make some noisy sound.